COMP3011
计算机体系结构

2.3 分支预测和前瞻执行及静态调度

授课教师：王强

邮箱：qiang.wang@hit.edu.cn



第2.3节 分支预测和前瞻执行及静态调度

1、分支预测
2、前瞻执行
3、静态调度

目录

➢ 分支预测

- 分支历史表
- 采用分支目标缓冲器

➢ 前瞻执行

- 基本概念和关键思想
- Tomasulo扩展实现

➢ 静态调度

- 基本指令调度
- 循环展开

1、分支预测 - 动态分支预测技术

控制相关（简单回顾）

- 控制相关是指由分支指令引起的相关。
  - 为了保证程序应有的执行顺序，必须严格按控制相关确定的顺序执行。
- 典型的程序结构是“if-then”结构。

```
if p1 {
        S1;
    };
S;
if p2 {
        S2;
    };
```

```
if p1 {
        S1;
    };
S;
if p2 {
        S2;
    };
```

- 控制相关带来了以下两个限制：
  - 与一条分支指令控制相关的指令不能被移到该分支之前。否则这些指令就不受该分支控制了。

    对于上述的例子，then 部分中的指令不能移到if语句之前。
  - 如果一条指令与某分支指令不存在控制相关，就不能把该指令移到该分支之后。

  对于上述的例子，不能把S移到if语句的then 部分中。

1. 控制相关并不是一个必须严格保持的关键属性。

2. 对于正确地执行程序来说，必须保持的最关键的两个属性是：数据流和异常行为。

- 保持异常行为是指：无论怎么改变指令的执行顺序，都不能改变程序中异常的发生情况。
  - 即原来程序中是怎么发生的，改变执行顺序后还是怎么发生。
  - 弱化为：指令执行顺序的改变不能导致程序中发生新的异常。

```
        DADDU    R2, R3, R4
        BEQZ     R2, L1
        LW       R1, 0(R2)
L1:
```

忽略控制相关而把LW指令移到分支指令之前，则有可能引起内存保护异常的错误。

➢数据流：指数据值从其产生者指令到其消费者指令的实际流动。

- 分支指令使得数据流具有动态性，因为一条指令有可能数据相关于多条先前的指令。
- 分支指令的执行结果决定了哪条指令真正是所需数据的产生者。

```
        DADDU   R1, R2, R3
        BEQZ    R4, L
        DSUBU   R1, R5, R6
L:      …
        OR      R7, R1, R8
```

- 有时，不遵守控制相关既不影响异常行为，也不改变数据流。
  - **可以大胆地进行指令调度，把失败分支中的指令调度到分支指令之前。**
  - **举例**

```
        DADDU      R1，R2，R3
        BEQZ       R12，Skipnext
        DSUBU      R4，R5，R6
        DADDU      R5，R4，R9
Skipnext：OR       R7，R8，R9
```

1、分支预测 - 动态分支预测技术

- 所开发的ILP越多，控制相关的制约就越大，分支预测就要有更高的准确度。
- 本节中介绍的方法对于每个时钟周期流出多条指令（若为n条，就称为n流出）的处理机来说非常重要。
  - 在n-流出的处理机中，遇到分支指令的可能性增加了n倍
  - 要给处理器连续提供指令，就需要准确地预测分支

1、分支预测

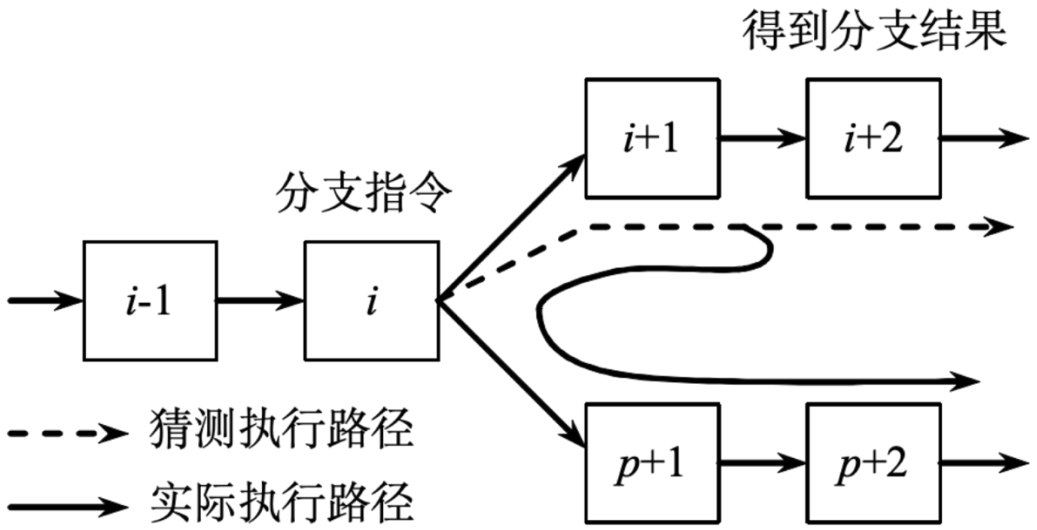
- 动态分支预测：在程序运行时，根据分支指令过去的表现来预测其将来的行为。
  - 如果分支行为发生了变化，预测结果也跟着改变。
  - 有更好的预测准确度和适应性。
- 分支预测的有效性取决于：
  - 预测的准确性
  - 预测正确和不正确两种情况下的分支开销
  - 决定分支开销的因素：
    - 流水线的结构
    - 预测的方法
    - 预测错误时的恢复策略等

1、分支预测

- 采用动态分支预测技术的目的
  - 预测分支是否成功
  - 尽快找到分支目标地址（或指令）

    （避免控制相关造成流水线停顿）
- 需要解决的关键问题
  - 如何记录分支的历史信息，要记录哪些信息？
  - 如何根据这些信息来预测分支的去向，甚至提前取出分支目标处的指令？

- 在预测错误时，要作废已经预取和分析的指令，恢复现场，并从另一条分支路径重新取指令。

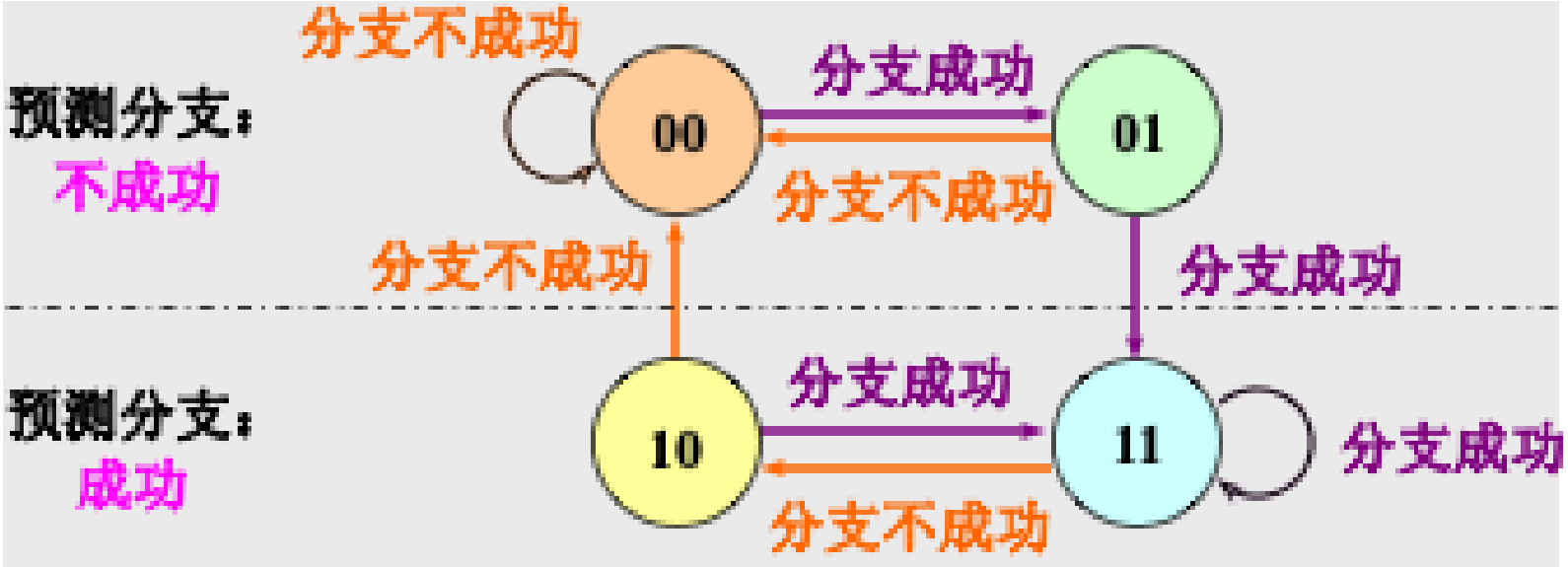


1、分支预测 - 分支历史表 BHT

- 分支历史表BHT（Branch History Table）
  - 最简单的动态分支预测方法。
  - 用BHT来记录分支指令最近一次或几次的执行情况（成功还是失败 ），并据此进行预测。
- 只有1个预测位的分支预测
  - 记录分支指令最近一次的历史，BHT中只需要1位二进制位。（最简单）

1、分支预测 - 分支历史表 BHT

- 采用两位二进制位来记录历史
  - 提高预测的准确度
  - 研究结果表明：两位分支预测的性能与n位（n>2）分支预测的性能差不多。
- 两位分支预测的状态转换如下所示：



1、分支预测 - 分支历史表 BHT

- 两位分支预测中的操作有两个步骤：
  - 分支预测；
    - 当分支指令到达译码段（ID）时，根据从BHT读出的信息进行分支预测 。
    - 若预测正确，就继续处理后续的指令，流水线没有断流。否则，就要作废已经预取和分析的指令，恢复现场，并从另一条分支路径重新取指令。
  - 状态修改。

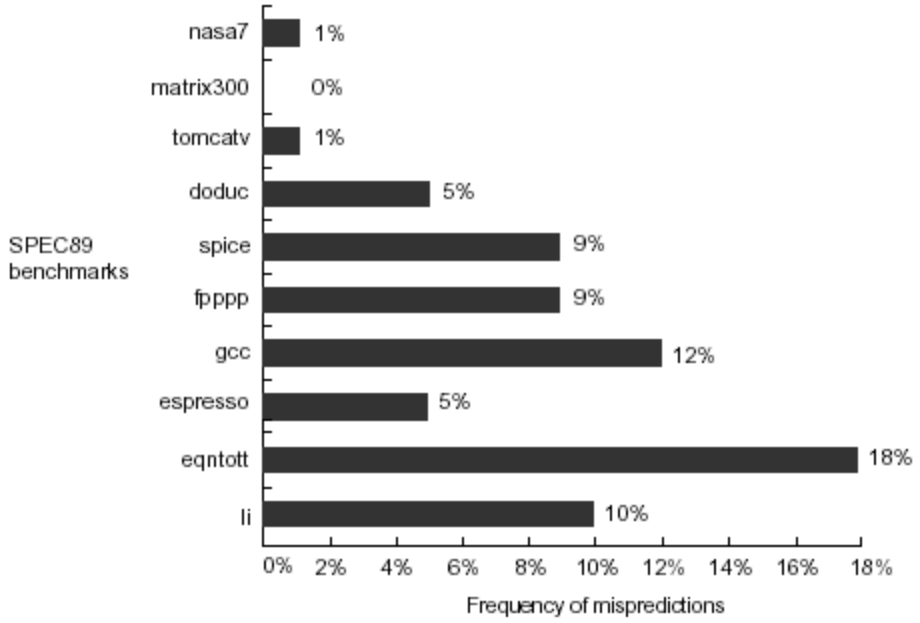
1、分支预测 - 分支历史表 BHT

- BHT可以跟分支指令一起存放在指令Cache中，也可以用一块专门的硬件来实现。
  - 可以作为一对指令位附于指令缓冲站的每一块中，并随指令一起读取。
  - 可以用存放指令地址的小的专用Cache来实现
    - 通过指令地址的低位访问BHT

1、分支预测 - 分支历史表 BHT

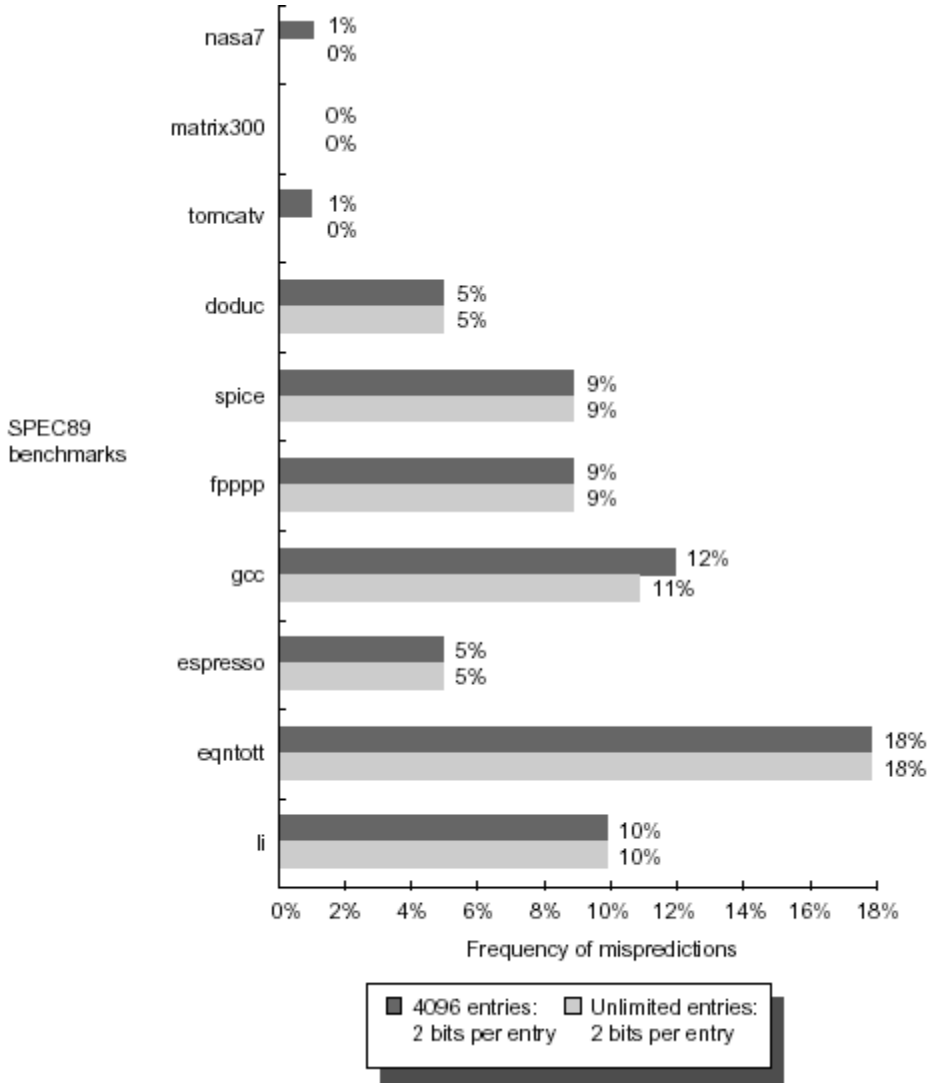
- 研究结果表明：对于SPEC89测试程序来说，具有大小为4K的BHT的预测准确率为82%～99%。

一般来说，采用4K的BHT就可以了。



- **每个条目包含**：
  - **分支目标地址**（Target Address）：记录预测跳转的目标地址
  - **历史状态位**（2-bit饱和计数器）：典型实现为"00=强不跳转"到"11=强跳转"
  - **标签位**（Tag Bits）：用于解决哈希冲突，通常为地址的高位部分

1、分支预测 - 分支历史表 BHT



1、分支预测 - 分支历史表 BHT

- BHT方法只在以下情况下才有用：
  - 判定分支是否成功所需的时间大于确定分支目标地址所需的时间。

    前述5段经典流水线：由于判定分支是否成功和计算分支目标地址都是在ID段完成，所以BHT方法不会给该流水线带来好处。

1、分支预测 - 分支目标缓冲器BTB

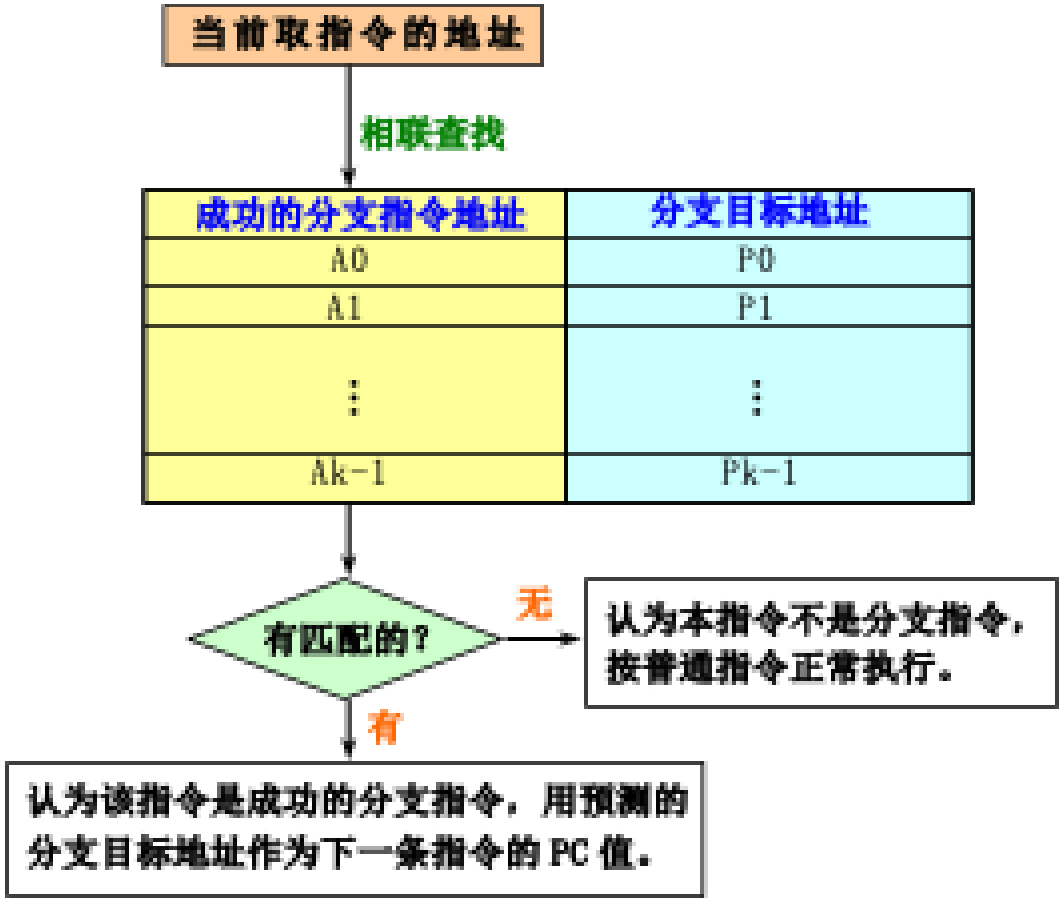
目标：将分支的开销降为 0

方法：分支目标缓冲

- 将分支成功的分支指令的地址和它的分支目标地址都放到一个缓冲区中保存起来，缓冲区以分支指令的地址作为标识。
- 这个缓冲区就是分支目标缓冲器（Branch-Target Buffer，简记为BTB，或者分支目标Cache （Branch-Target Cache）。

1、分支预测 - 分支目标缓冲器BTB

- BTB的结构

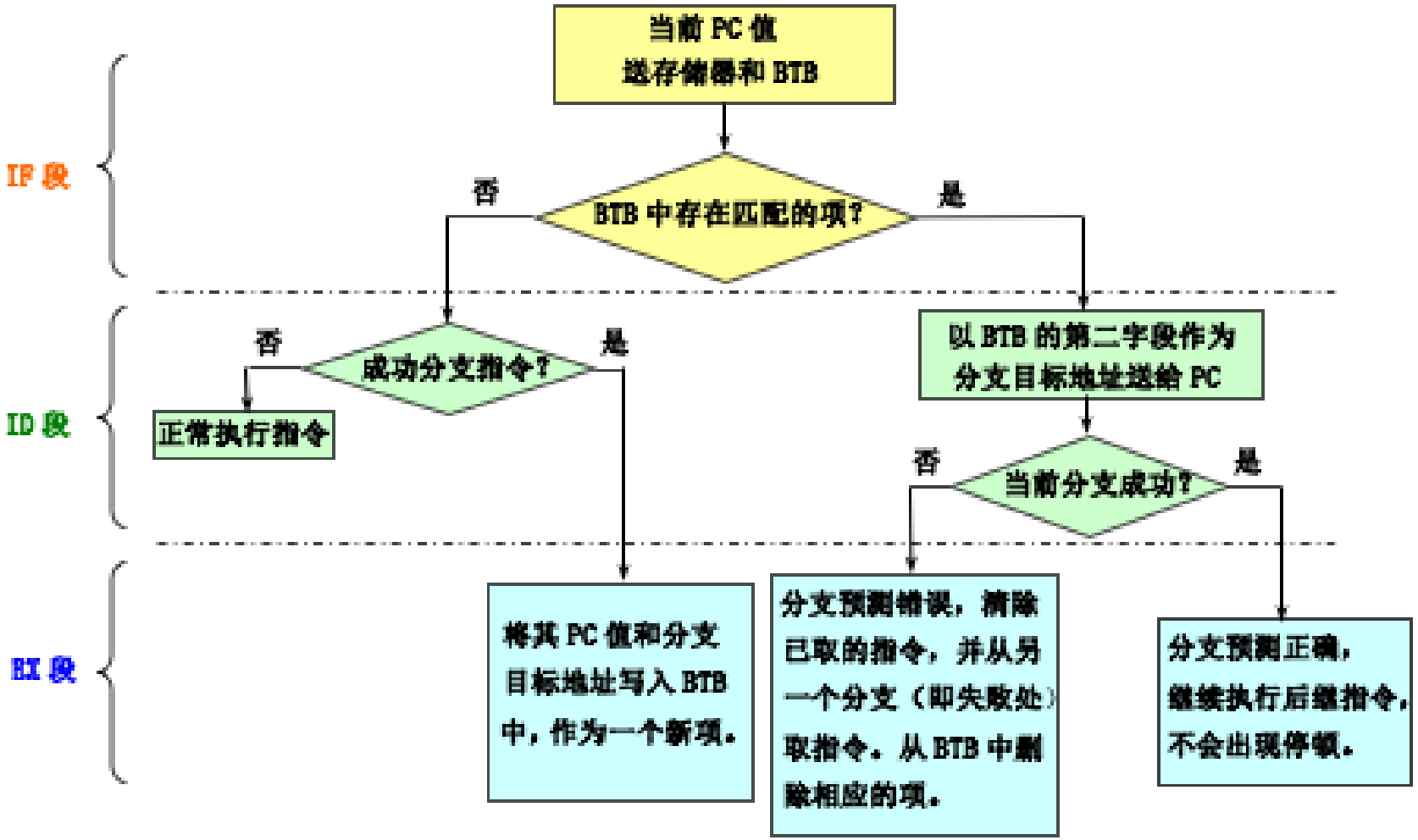


1、分支预测 - 分支目标缓冲器BTB

- BTB是用专门的硬件实现的一张表格。
- 表格中的每一项至少有两个字段：
  - 执行过的成功分支指令的地址；
    （作为该表的匹配标识 ）
  - 预测的分支目标地址。

1、分支预测 - 分支目标缓冲器BTB

采用BTB后，在流水线各个阶段所进行的相关操作：



1、分支预测 - 分支目标缓冲器BTB

➢ 采用BTB后，各种可能情况下的延迟 ：

指令在BTB中？	预测	实际情况	延迟周期
是	成功	成功	0
是	成功	不成功	2
不是		成功	2
不是		不成功	0

1、分支预测 - 分支目标缓冲器BTB

Example Determine the total branch penalty for a branch-target buffer assuming the penalty cycles for individual mispredictions from Figure 3.23. Make the following assumptions about the prediction accuracy and hit rate:

- Prediction accuracy is 90% (for instructions in the buffer).
- Hit rate in the buffer is 90% (for branches predicted taken).

Answer We compute the penalty by looking at the probability of two events: the branch is predicted taken but ends up being not taken, and the branch is taken but is not found in the buffer. Both carry a penalty of two cycles.

$$\begin{aligned}
\text{Probability (branch in buffer, but actually not taken)} &= \text{Percent buffer hit rate} \times \text{Percent incorrect predictions} \\
&= 90\% \times 10\% = 0.09 \\
\text{Probability (branch not in buffer, but actually taken)} &= 10\% \\
\text{Branch penalty} &= (0.09 + 0.10) \times 2 \\
\text{Branch penalty} &= 0.38
\end{aligned}$$

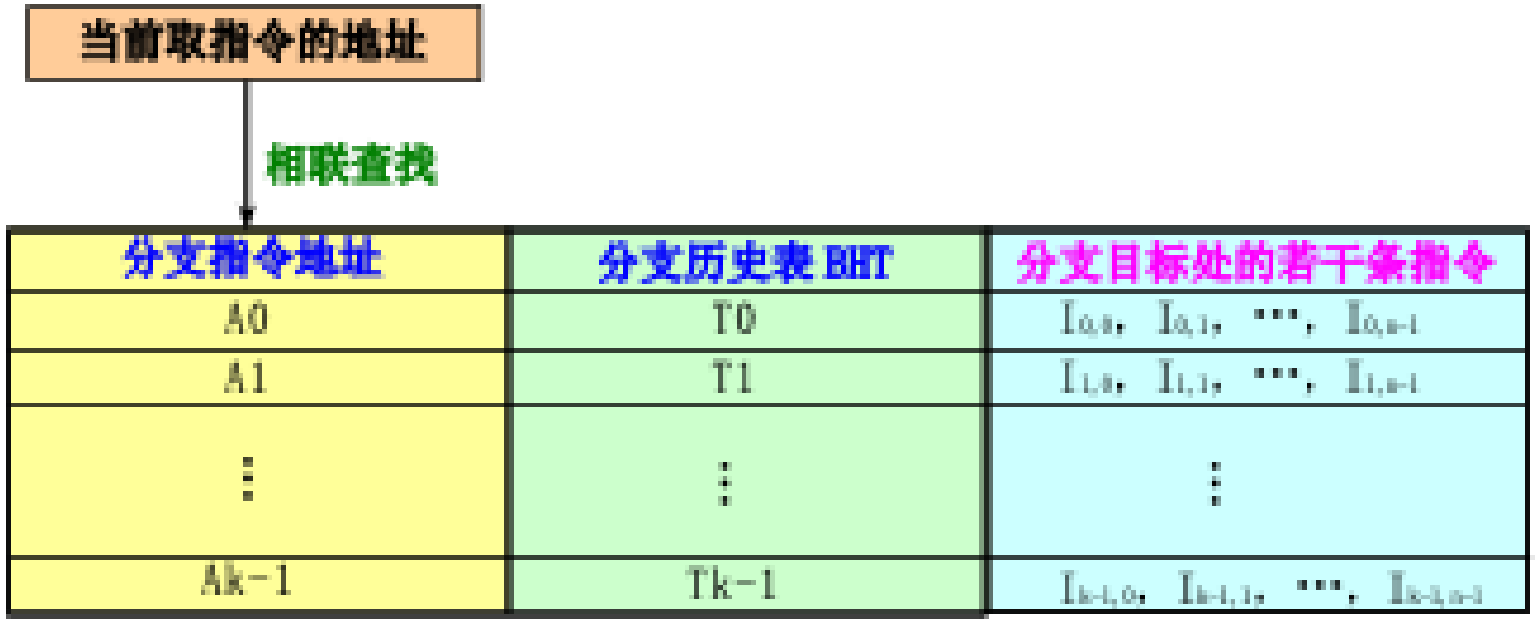
1、分支预测 - 分支目标缓冲器BTB

- BTB的另一种形式
  - 在分支目标缓冲器中增设一个至少是两位的“分支历史表”字段



1、分支预测 - 分支目标缓冲器BTB

- 更进一步，在表中对于每条分支指令都存放若干条分支目标处的指令，就形成了分支目标指令缓冲器。



- 允许**分支目标缓冲站访问时间超过相继指令间的取指令时间**
  - 从而允许使用更大的分支目标缓冲站
- 分支折叠

2、前瞻执行

前瞻执行（speculation）的基本思想：

对分支指令的结果进行猜测，并假设这个猜测总是对的，然后按这个猜测结果继续取、流出和执行后续的指令。只是执行指令的结果不是写回到寄存器或存储器，而是写入一个称为再定序缓冲器ROB（ReOrder Buffer）中 。等到相应的指令得到“确认”（commit）（即确实是应该执行的）之后，才将结果写入寄存器或存储器。

2、前瞻执行

1. 基于硬件的前瞻执行结合了3种思想：
   - 动态分支预测。用来选择后续执行的指令。
   - 在控制相关的结果尚未出来之前，前瞻地执行后续指令。
   - 用动态调度对基本块的各种组合进行跨基本块的调度。
2. 对Tomasulo算法加以扩充，就可以支持前瞻执行。

   把Tomasulo算法的写结果和指令完成加以区分，分成两个不同的段：

   写结果，指令确认

   已经在一系列处理机上实现了（**PowerPC 620，MIPS R10000，Intel P6和AMD K5**）

2、前瞻执行

- 写结果段
  - 把前瞻执行的结果写到ROB中；
  - 通过CDB在指令之间传送结果，供需要用到这些结果的指令使用。
- 指令确认段

  在分支指令的结果出来后，对相应指令的前瞻执行给予确认。
  - 如果前面所做的猜测是对的，把在ROB中的结果写到寄存器或存储器。
  - 如果发现前面对分支结果的猜测是错误的，那就不予以确认，并从那条分支指令的另一条路径开始重新执行。

2、前瞻执行

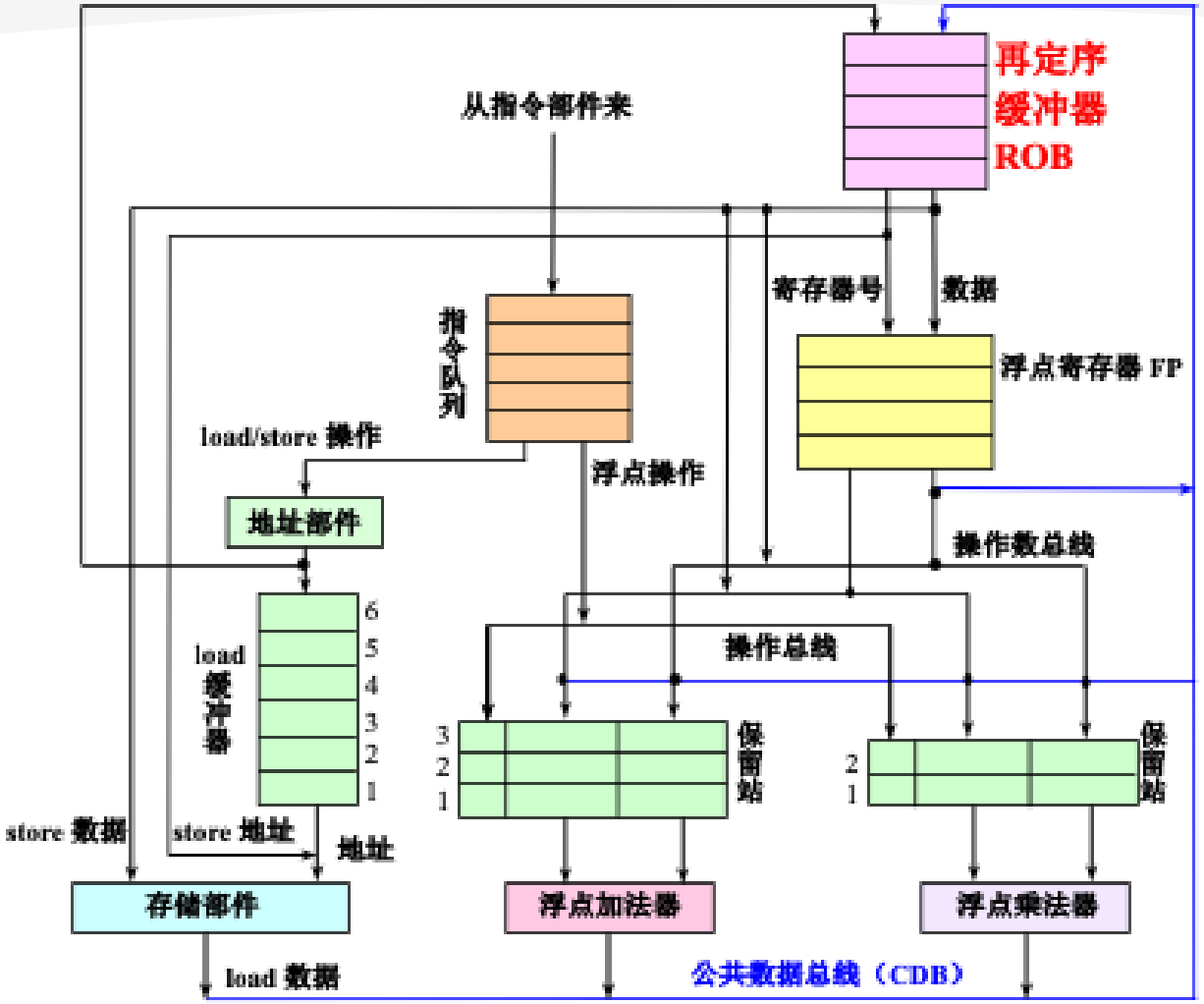
- 实现前瞻的关键思想：

  允许指令乱序执行，但必须顺序确认。

  在指令被确认之前，不允许它进行不可恢复的操作。

2、前瞻执行

支持前瞻执行的浮点部件的结构



2、前瞻执行

- ROB中的每一项由以下4个字段组成:
  - 指令类型

    指出该指令是分支指令、store指令或寄存器操作指令。
  - 目标地址

    给出指令执行结果应写入的目标寄存器号（如果是load和ALU指令）或存储器单元的地址（如果是store指令）。
  - 数据值字段

    用来保存指令前瞻执行的结果，直到指令得到确认。
  - 就绪字段

    指出指令是否已经完成执行并且数据已就绪。

Tomasulo算法中保留站的换名功能是由ROB来完成的。

2、前瞻执行

- 采用前瞻执行机制后，指令的执行步骤：

（在Tomasulo算法的基础上改造的）

- 流出
  - 从浮点指令队列的头部取一条指令。
  - 如果有空闲的保留站（设为r）且有空闲的ROB项（设为b），就流出该指令，并把相应的信息放入保留站r和ROB项b。
    - 如果寄存器或者ROB中已经含有源操作数，将其发入保留站r，否则将产生该源操作数的指令所分配的ROB编号送入r
    - 将为该指令分配的保留站编号送入r
  - 如果保留站或ROB全满，便停止流出指令，直到它们都有空闲的项。

2、前瞻执行

- 执行
  - 如果有操作数尚未就绪，就等待，并不断地监测CDB。

    （检测RAW冲突）
  - 当两个操作数都已在保留站中就绪后，就可以执行该指令的操作。

2、前瞻执行

➢ 写结果

- 当结果产生后，将该结果连同本指令在流出段所分配到的ROB项的编号放到CDB上，经CDB写到ROB以及所有等待该结果的保留站。
- 释放产生该结果的保留站。
- store指令在本阶段完成操作为:
  - 如果要写入存储器的数据已经就绪，就把该数据写入分配给该store指令的ROB项。
  - 否则，就监测CDB，直到那个数据在CDB上播送出来，才将之写入分配给该store指令的ROB项。

2、前瞻执行

➢ 确认

对分支指令、store指令以及其它指令的处理不同：

- 其它指令（除分支指令和store指令）

  当该指令到达ROB队列的头部而且其结果已经就绪时，就把该结果写入该指令的目的寄存器，并从ROB中删除该指令。

- store指令

  处理与上面的类似，只是它把结果写入存储器。

2、前瞻执行

- 分支指令
  - 当预测错误的分支指令到达ROB队列的头部时，清空ROB，并从分支指令的另一个分支重新开始执行。

    （错误的前瞻执行）
  - 当预测正确的分支指令到达ROB队列的头部时，该指令执行完毕。

2、前瞻执行

例2.4 假设浮点功能部件的延迟时间为：加法2个时钟周期，乘法10个时钟周期，除法40个时钟周期。对于下面的代码段，给出当指令MUL.D即将确认时的状态表内容。

```
L.D     F6,34（R2）
L.D     F2,45（R3）
MUL.D   F0,F2,F4
SUB.D   F8,F6,F2
DIV.D   F10,F0,F6
ADD.D   F6,F8,F2
```

Cycle 0

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2							

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Head →
Tail →

Entry	Busy	Instruction	Stage	Dest	Value
1					
2					
3					
4					
5					
6					
7					

指令		指令状态表			
		流出	执行	写结果	确认
L.D	F6, 34(R2)				
L.D	F2, 45(R3)				
MULT.D	F0, F2, F4				
SUB.D	F8, F6, F2				
DIV.D	F10, F0, F6				
ADD.D	F6, F8, F2				

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号								
Busy								
Value								

Cycle 1

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2							

	Load保留站	
	Busy	Address
Load1	Yes	34+R2
Load2		
Load3		

Head

Tail

Entry	Busy	Instruction	Stage	Dest	Value
1	Yes	L.D F6, 34(R2)	Issue	R6	
2					
3					
4					
5					
6					
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1			
L.D F2, 45(R3)				
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号				#1				
Busy				Yes				
Value								

Cycle 2

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2							

	Load保留站	
	Busy	Address
Load1	Yes	34+R2
Load2	Yes	45+R3
Load3		

Head →
Tail →

Entry	Busy	Instruction	Stage	Dest	Value
1	Yes	L.D F6, 34(R2)	Ex	R6	
2	Yes	L.D F2, 45(R3)	Issue	R2	
3					
4					
5					
6					
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2		
L.D F2, 45(R3)	2			
MULT.D F0, F2, F4				
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号		#2		#1				
Busy		Yes		Yes				
Value								

Cycle 3

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1	Yes	MUL		V(R4)	#2		#3
Mul2							

	Load保留站	
	Busy	Address
Load1	Yes	34+R2
Load2	Yes	45+R3
Load3		

Head →

Tail →

Entry	Busy	Instruction	Stage	Dest	Value
1	Yes	L.D F6, 34(R2)	Ex	R6	
2	Yes	L.D F2, 45(R3)	Ex	R2	
3	Yes	MULT.D F0, F2, F4	Issue	R0	
4					
5					
6					
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3		
L.D F2, 45(R3)	2	3		
MULT.D F0, F2, F4	3			
SUB.D F8, F6, F2				
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3	#2		#1				
Busy	Yes	Yes		Yes				
Value								

Cycle 4

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1	Yes	SUB	M1			#2	#4
Add2							
Add3							
Mul1	Yes	MUL		V(R4)	#2		#3
Mul2							

	Load保留站	
	Busy	Address
Load1	~~Yes~~	~~34+R2~~
Load2	Yes	45+R3
Load3		

Head →

Tail →

Entry	Busy	Instruction	Stage	Dest	Value
1	Yes	L.D F6, 34(R2)	WB	R6	M1
2	Yes	L.D F2, 45(R3)	EX	R2	
3	Yes	MULT.D F0, F2, F4	Issue	R0	
4	Yes	SUB.D F8, F6, F2	Issue	R8	
5					
6					
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	
L.D F2, 45(R3)	2	3-4		
MULT.D F0, F2, F4	3			
SUB.D F8, F6, F2	4			
DIV.D F10, F0, F6				
ADD.D F6, F8, F2				

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3	#2		#1	#4			
Busy	Yes	Yes		Yes	Yes			
Value								

Cycle 5

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1	Yes	SUB	M1	M2		~~#2~~	#4
Add2							
Add3							
Mul1	Yes	MUL	M2	V(R4)	~~#2~~		#3
Mul2	Yes	DIV		M1	#3		#5

	Load保留站	
	Busy	Address
Load1		
Load2	~~Yes~~	~~45+R3~~
Load3		

	Entry	Busy	Instruction	Stage	Dest	Value
Head →	1	No	L.D F6, 34(R2)	Commit	R6	M1
	2	Yes	L.D F2, 45(R3)	WB	R2	M2
	3	Yes	MULT.D F0, F2, F4	Issue	R0	
	4	Yes	SUB.D F8, F6, F2	Issue	R8	
Tail →	5	Yes	DIV.D F10, F0, F6	Issue	R10	
	6					
	7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	
MULT.D F0, F2, F4	3			
SUB.D F8, F6, F2	4			
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2				

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3	#2		~~#1~~	#4	#5		
Busy	Yes	Yes		~~Yes~~	Yes	Yes		
Value				M1				

Cycle 6

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1	Yes	SUB	M1	M2			#4
Add2	Yes	ADD		M2	#4		#6
Add3							
Mul1	Yes	MUL	M2	V(R4)			#3
Mul2	Yes	DIV		M1	#3		#5

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
Head → 3	Yes	MULT.D F0, F2, F4	EX	R0	
4	Yes	SUB.D F8, F6, F2	EX	R8	
5	Yes	DIV.D F10, F0, F6	Issue	R10	
Tail → 6	Yes	ADD.D F6, F8, F2	Issue	R6	
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6		
SUB.D F8, F6, F2	4	6		
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	6			

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3	~~#2~~		#6	#4	#5		
Busy	Yes	~~Yes~~		Yes	Yes	Yes		
Value		M2		M1				

Cycle 7

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1	Yes	SUB	M1	M2			#4
Add2	Yes	ADD		M2	#4		#6
Add3							
Mul1	Yes	MUL	M2	V(R4)			#3
Mul2	Yes	DIV		M1	#3		#5

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
Head → 3	Yes	MULT.D F0, F2, F4	EX	R0	
4	Yes	SUB.D F8, F6, F2	EX	R8	
5	Yes	DIV.D F10, F0, F6	Issue	R10	
Tail → 6	Yes	ADD.D F6, F8, F2	Issue	R6	
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-7		
SUB.D F8, F6, F2	4	6-7		
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	6			

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3			#6	#4	#5		
Busy	Yes			Yes	Yes	Yes		
Value		M2		M1				

Cycle 8

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1	~~Yes~~	~~SUB~~	~~M1~~	~~M2~~			~~#4~~
Add2	Yes	ADD	Val1	M2	~~#4~~		#6
Add3							
Mul1	Yes	MUL	M2	V(R4)			#3
Mul2	Yes	DIV		M1	#3		#5

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
Head → 3	Yes	MULT.D F0, F2, F4	EX	R0	
4	Yes	SUB.D F8, F6, F2	WB	R8	Val1
5	Yes	DIV.D F10, F0, F6	Issue	R10	
Tail → 6	Yes	ADD.D F6, F8, F2	Issue	R6	
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-8		
SUB.D F8, F6, F2	4	6-7	8	
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	6			

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3			#6	#4	#5		
Busy	Yes			Yes	Yes	Yes		
Value		M2		M1				

Cycle 9

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2	Yes	ADD	Val1	M2			#6
Add3							
Mul1	Yes	MUL	M2	V(R4)			#3
Mul2	Yes	DIV		M1	#3		#5

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
Head → 3	Yes	MULT.D F0, F2, F4	EX	R0	
4	Yes	SUB.D F8, F6, F2	WB	R8	Val1
5	Yes	DIV.D F10, F0, F6	Issue	R10	
Tail → 6	Yes	ADD.D F6, F8, F2	EX	R6	
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-8		
SUB.D F8, F6, F2	4	6-7	8	
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	6	9		

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3			#6	#4	#5		
Busy	Yes			Yes	Yes	Yes		
Value		M2		M1				

Cycle 11

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2	Yes	ADD	Val1	M2			#6
Add3							
Mul1	Yes	MUL	M2	V(R4)			#3
Mul2	Yes	DIV		M1	#3		#5

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
Head → 3	Yes	MULT.D F0, F2, F4	EX	R0	
4	Yes	SUB.D F8, F6, F2	WB	R8	Val1
5	Yes	DIV.D F10, F0, F6	Issue	R10	
Tail → 6	Yes	ADD.D F6, F8, F2	WB	R6	Val2
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-11		
SUB.D F8, F6, F2	4	6-7	8	
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	6	9-10	11	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3			#6	#4	#5		
Busy	Yes			Yes	Yes	Yes		
Value		M2		M1				

Cycle 16

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1	~~Yes~~	~~MUL~~	~~M2~~	~~V(R4)~~			~~#3~~
Mul2	Yes	DIV	Val3	M1	~~#3~~		#5

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
Head → 3	Yes	MULT.D F0, F2, F4	WB	R0	Val3
4	Yes	SUB.D F8, F6, F2	WB	R8	Val1
5	Yes	DIV.D F10, F0, F6	Issue	R10	
Tail → 6	Yes	ADD.D F6, F8, F2	WB	R6	Val2
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-15	16	
SUB.D F8, F6, F2	4	6-7	8	
DIV.D F10, F0, F6	5			
ADD.D F6, F8, F2	6	9-10	11	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	#3			#6	#4	#5		
Busy	Yes			Yes	Yes	Yes		
Value		M2		M1				

Cycle 17

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2	Yes	DIV	Val3	M1	#3		#5

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
3	No	MULT.D F0, F2, F4	Commit	R0	Val3
4	Yes	SUB.D F8, F6, F2	WB	R8	Val1
5	Yes	DIV.D F10, F0, F6	EX	R10	
6	Yes	ADD.D F6, F8, F2	WB	R6	Val2
7					

Head → 4

Tail → 6

指令		指令状态表			
		流出	执行	写结果	确认
L.D	F6, 34(R2)	1	2-3	4	5
L.D	F2, 45(R3)	2	3-4	5	6
MULT.D	F0, F2, F4	3	6-15	16	17
SUB.D	F8, F6, F2	4	6-7	8	
DIV.D	F10, F0, F6	5	17		
ADD.D	F6, F8, F2	6	9-10	11	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号	~~#3~~			#6	#4	#5		
Busy	~~Yes~~			Yes	Yes	Yes		
Value	Val3	M2		M1				

Cycle 18

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2	Yes	DIV		M1	#3		#5

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
3	No	MULT.D F0, F2, F4	Commit	R0	Val3
4	No	SUB.D F8, F6, F2	Commit	R8	Val1
Head → 5	Yes	DIV.D F10, F0, F6	EX	R10	
Tail → 6	Yes	ADD.D F6, F8, F2	WB	R6	Val2
7					

指令		指令状态表			
		流出	执行	写结果	确认
L.D	F6, 34(R2)	1	2-3	4	5
L.D	F2, 45(R3)	2	3-4	5	6
MULT.D	F0, F2, F4	3	6-15	16	17
SUB.D	F8, F6, F2	4	6-7	8	18
DIV.D	F10, F0, F6	5	17-18		
ADD.D	F6, F8, F2	6	9-10	11	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号				#6	~~#4~~	#5		
Busy				Yes	~~Yes~~	Yes		
Value	Val3	M2		M1	Val1			

Cycle 57

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2	~~Yes~~	~~DIV~~		~~M1~~	~~#3~~		~~#5~~

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
3	No	MULT.D F0, F2, F4	Commit	R0	Val3
4	No	SUB.D F8, F6, F2	Commit	R8	Val1
Head → 5	Yes	DIV.D F10, F0, F6	WB	R10	Val4
Tail → 6	Yes	ADD.D F6, F8, F2	WB	R6	Val2
7					

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-15	16	17
SUB.D F8, F6, F2	4	6-7	8	18
DIV.D F10, F0, F6	5	17-56	57	
ADD.D F6, F8, F2	6	9-10	11	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号				#6		#5		
Busy				Yes		Yes		
Value	Val3	M2		M1	Val1			

Cycle 58

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2							

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
3	No	MULT.D F0, F2, F4	Commit	R0	Val3
4	No	SUB.D F8, F6, F2	Commit	R8	Val1
5	No	DIV.D F10, F0, F6	Commit	R10	Val4
6	Yes	ADD.D F6, F8, F2	WB	R6	Val2
7					

Head

Tail

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-15	16	17
SUB.D F8, F6, F2	4	6-7	8	18
DIV.D F10, F0, F6	5	17-56	57	58
ADD.D F6, F8, F2	6	9-10	11	

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号				#6				
Busy				Yes				
Value	Val3	M2		M1	Val1	Val4		

Cycle 59

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2							

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
3	No	MULT.D F0, F2, F4	Commit	R0	Val3
4	No	SUB.D F8, F6, F2	Commit	R8	Val1
5	No	DIV.D F10, F0, F6	Commit	R10	Val4
6	No	ADD.D F6, F8, F2	Commit	R6	Val2
7					

Head

Tail

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-15	16	17
SUB.D F8, F6, F2	4	6-7	8	18
DIV.D F10, F0, F6	5	17-56	57	58
ADD.D F6, F8, F2	6	9-10	11	59

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号				~~#6~~				
Busy				~~Yes~~				
Value	Val3	M2		Val2	Val1	Val4		

总结

部件名称	执行部件保留站						
	Busy	Op	Vj	Vk	Qj	Qk	Dest
Add1							
Add2							
Add3							
Mul1							
Mul2							

	Load保留站	
	Busy	Address
Load1		
Load2		
Load3		

Entry	Busy	Instruction	Stage	Dest	Value
1	No	L.D F6, 34(R2)	Commit	R6	M1
2	No	L.D F2, 45(R3)	Commit	R2	M2
3	No	MULT.D F0, F2, F4	Commit	R0	Val3
4	No	SUB.D F8, F6, F2	Commit	R8	Val1
5	No	DIV.D F10, F0, F6	Commit	R10	Val4
6	No	ADD.D F6, F8, F2	Commit	R6	Val2
7					

Head

Tail

指令	指令状态表			
	流出	执行	写结果	确认
L.D F6, 34(R2)	1	2-3	4	5
L.D F2, 45(R3)	2	3-4	5	6
MULT.D F0, F2, F4	3	6-15	16	17
SUB.D F8, F6, F2	4	6-7	8	18
DIV.D F10, F0, F6	5	17-56	57	58
ADD.D F6, F8, F2	6	9-10	11	59

按序发射　乱序执行　按序提交

	结果寄存状态表							
	F0	F2	F4	F6	F8	F10	...	F30
ROB编号								
Busy								
Value	Val3	M2		Val2	Val1	Val4		

3、前瞻执行 - 基于硬件

前瞻执行中MUL.D确认前，保留站和ROB的状态

名称	保留站							
	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	no							
Load2	no							
Add1	no							
Add2	no							
Add3	no							
Mult1	no	MUL.D	Mem[45+ Regs[R2]]	Regs[F4]			#3	
Mult2	yes	DIV.D		Mem[34+Regs[R2]]	#3		#5	

项号	ROB				
	Busy	指令	状态	目的	Value
1	no	L.D F6, 34（R2）	确认	F6	Mem[34+Regs[R2]]
2	no	L.D F2, 45（R3）	确认	F2	Mem[45+Regs[R3]]
3	yes	MUL.D F0, F2, F4	写结果	F0	#2×Regs[F4]
4	yes	SUB.D F8, F6, F2	写结果	F8	#1－#2
5	yes	DIV.D F10, F0, F6	执行	F10	
6	yes	ADD.D F6,F8,F2	写结果	F6	#4＋#2

字段	浮点寄存器状态							
	F0	F2	F4	F6	F8	F10	…	F30
ROB项编号	3			6	4	5		
Busy	yes	no	no	yes	yes	yes	…	no

2、前瞻执行

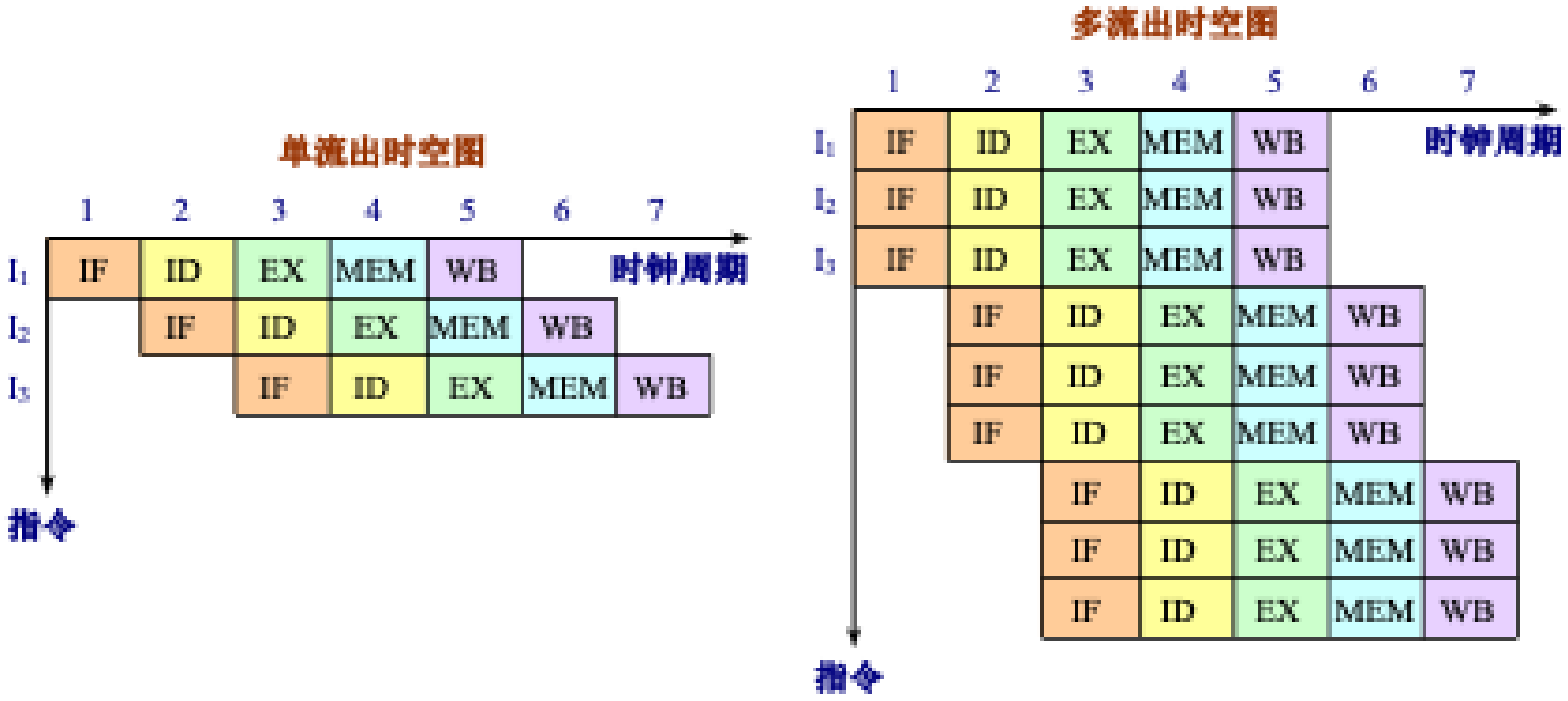
前瞻执行

- 通过ROB实现了指令的顺序完成。
  - 能够实现精确异常。
- 很容易地推广到整数寄存器和整数功能单元上。
- 可应用于多发射处理器
- 主要缺点：所需的硬件太复杂。

扩展：多指令流出

利用多指令流出获得更高的ILP

- 在每个时钟周期内流出多条指令， CPI＜1。
- 单流出和多流出处理机执行指令的时空图对比



单流出和多流出处理机执行指令的时空图

扩展：多指令流出

- 多流出处理机有两种基本风格：
  - 超标量（Superscalar）
    - 在每个时钟周期流出的指令条数不固定，依代码的具体情况而定。（有个上限）
    - 设这个上限为n，就称该处理机为n-流出。
    - 可以通过编译器进行静态调度，也可以基于Tomasulo算法进行动态调度。
  - 超长指令字VLIW（Very Long Instruction Word）
    - 在每个时钟周期流出的指令条数是固定的，这些指令构成一条长指令或者一个指令包。
    - 指令包中，指令之间的并行性是通过指令显式地表示出来的。
    - 指令调度是由编译器静态完成的。

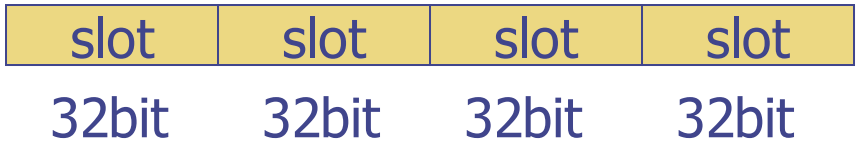
扩展阅读-静态多指令流出：VLIW技术

1. VLIW vs. 超标量

- 在动态调度的超标量处理器中，相关检测和指令调度基本都由硬件完成。
- 在静态调度的超标量处理器中，部分相关检测和指令调度工作交由编译器完成。
- 在VLIW处理器中，相关检测和指令调度工作全部由编译器完成，它需要更“智能”的编译器。
  - 编译器要保证在一个发射包里的指令是不相关的，或者至少指明何时会发生相关
  - 硬件不必再显式地检查相关性

2. VLIW

- 第一代多发射处理器采用了长指令的格式，在一条长指令中包含有多个操作
  - 指令的长度往往很长（**64~128**位或者更长）
  - 将没有相关的操作封装成一条长指令或在编译时将它们标识为发射包
- VLIW的性能会随着最高发射速率的增加而提高
  - 发射速率越大，超标量的开销就越大**-4**发射
  - **VLIW**对于发射速率较大的处理器才有意义



128bit VLIW指令

3. 实例分析

例 假设某VLIW处理器每个时钟周期可以同时流出5个操作，包括2个访存操作，2个浮点操作以及1个整数或分支操作。将前面例子中的代码循环展开，并调度到该VLIW处理器上执行。循环展开次数不定，但至少要能够保证消除所有流水线“暂停”周期，同时不考虑分支延迟。

- 为了保持功能部件忙碌，代码中必须存在**足够大的并行性**，以将**不相关的操作安放到相应的指令槽**中。
- 假设可以产生一段比较长的线性代码，能够使用局部指令调度来生成VLIW指令

访存操作1	访存操作2	浮点操作1	浮点操作2	整数分支操作
L.D F0，0（R1）	L.D F6，-8（R1）	nop	nop	nop
L.D F10，-16（R1）	L.D F14，-24（R1）	nop	nop	nop
L.D F18，-32（R1）	L.D F22，-40（R1）	ADD.D F4，F0，F2	ADD.D F8，F6，F2	nop
L.D F26，-48（R1）	nop	ADD.D F12，F10，F2	ADD.D F16，F14，F2	nop
nop	nop	ADD.D F20，F18，F2	ADD.D F24，F22，F2	nop
S.D 0（R1），F4	S.D -8（R1），F8	ADD.D F28，F26，F2	nop	nop
S.D -16（R1），F12	S.D -24（R1），F16	nop	nop	nop
S.D -32（R1），F20	S.D -40（R1），F24	nop	nop	DADDUI R1，R1，#56
S.D 8（R1），F28	nop	nop	nop	BNE R1，R2，Loop

解：循环被展开7次，经调度后可以消除所有流水线“暂停”。在不考虑分支延迟的情况下，每执行一个叠代需要9个时钟周期，计算出7个结果，即平均每得到一个结果需要1.29个周期。

VLIW的不足：

- 操作部件的利用率仅比50%略高一些
  - 9个周期只发射了23个操作
- 所需要的寄存器数量也大大增加

3. VLIW性能分析

- VLIW目标代码编码效率低
  - 为消除流水线“暂停”需要增加循环展开的次数
  - 很难从应用程序中找到足够多的并行指令填满VLIW指令中的每一个slot
  - 指令压缩
- VLIW流水线的互锁机制
  - VLIW处理器中没有实现任何相关检测逻辑，而是靠互锁机制保证执行结果的正确
  - 这种简单的互锁机制将造成较大的开销
- 目标代码兼容性差
  - 二进制翻译
  - 放宽VLIW的限制以保持二进制兼容性（IA-64）

4. 性能比较——多流出处理器 vs. 向量处理器

- 即使对于一些结构不规则的代码，多流出处理器也能从中挖掘出一些指令级并行。
- 多流出处理器对存储系统没有过高的要求，价格较便宜、由Cache和主存构成的多层次存储子系统即可满足其对性能的要求。

结论：多流出处理器已成为当前实现指令级并行的主要选择，而向量处理器则通常是作为协处理器集成到计算机系统中，以加速特定类型的应用程序。

扩展：多指令流出

- 超标量处理机与VLIW处理机相比有两个优点：
  - 超标量结构对程序员是透明的，处理机能自己检测下一条指令能否流出，不需要由编译器或专门的变换程序对程序中的指令进行重新排列；
  - 即使是没有经过编译器针对超标量结构进行调度优化的代码或是旧的编译器生成的代码也可以运行，当然运行的效果不会很好。
    - 要想达到很好的效果，方法之一：使用动态超标量调度技术。

技 术	流出结构	冲突检测	调 度	主要特点	处理机实例
超标量（静态）	动态	硬件	静态	按序执行	Sun UltraSPARCII/III
超标量（动态）	动态	硬件	动态	部分乱序执行	IBM Power2
超标量（前瞻）	动态	硬件	带有前瞻的动态调度	带有前瞻的乱序执行	Pentium III/4, MIPS R10K, Alpha 21264, HP PA 8500, IBM RS64III
VLIW /LIW	静态	软件	静态	流出包指令之间没有冲突	Trimedia，i860
EPIC	主要是静态	主要是软件	主要是静态	相关性被编译器显式地标记出来	Itanium

扩展：多指令流出-基于动态调度

- 扩展Tomasulo算法：支持双流出超标量流水线
  - 每个时钟周期流出两条指令；
  - 一条是整数指令，另一条是浮点指令。
- 采用一种比较简单的方法：
  - 指令按顺序流向保留站
  - 将整数所用的表结构与浮点用的表结构分离开，分别进行处理，这样就可以同时地流出一条浮点指令和一条整数指令到各自的保留站。除非这两条指令访问了同一个寄存器而无法同时发射

扩展：多指令流出-基于动态调度

1. 有两种不同的方法可以实现多流出（采用动态调度的机制上）

关键在于：对保留站的分配和对流水线控制表格的修改。

- 在半个时钟周期里完成流出步骤，这样一个时钟周期就能处理两条指令。
- 设置一次能同时处理两条指令的逻辑电路。

现代的流出4条或4条以上指令的超标量处理机经常是两种方法都采用。

扩展：多指令流出-基于动态调度

例2.5 对于采用了Tomasulo算法和多流出技术的MIPS流水线，考虑以下简单循环的执行。该程序把F2中的标量加到一个向量的每个元素上。

```
Loop: L.D     F0, 0(R1)        // 取一个数组元素放入F0
      ADD.D   F4, F0, F2       // 加上在F2中的标量
      S.D     F4, 0(R1)        // 存结果
      DADDIU  R1,R1, #-8       // 将指针减少8（每个数据占8个字节）
      BNE  R1,R2,Loop          // 若R1不等于R2，表示尚未结束，转移到Loop继续。
```

扩展：多指令流出-基于动态调度

现做以下假设：

- 每个时钟周期能流出一条整数指令和一条浮点指令，即使它们相关也是如此。
- 有一个整数部件，用于整数ALU运算和地址计算；并且对于每一种浮点操作类型都有一个独立的流水化了的浮点功能部件。
- 指令流出和写结果各占用一个时钟周期。
- 跟大多数动态调度处理器一样，写结果段（**CDB**）的存在意味着实际的指令延迟会比按序流动的简单流水线多一个时钟周期。
- 具有动态分支预测部件和一个独立的计算分支条件的功能部件。

扩展：多指令流出-基于动态调度

1. 请列出该程序前面3遍循环中各条指令的流出、开始执行和将结果写到CDB上的时间。
2. 如果分支指令单流出，没有采用延迟分支，但分支预测是完美的。请列出整数部件、浮点部件、数据Cache以及CDB的资源使用情况。

解 执行时，该循环将动态展开，并且只要可能就流出两条指令。表中列出了各指令执行到几个操作点的时间及资源的使用情况。

遍数	指　　令		流出	执行	访存	写CDB	说明
1	L.D	F0,0（R1）	1	2	3	4	流出第一条指令
1	ADD.D	F4,F0,F2	1	5		8	等待L.D的结果
1	S.D	F4,0（R1）	2	3	9		等待ADD.D的结果
1	DADDIU	R1,R1,#-8	2	4		5	等待ALU
1	BNE	R1,R2,Loop	3	6			等待DADDIU的结果
2	L.D	F0,0（R1）	4	7	8	9	等待BNE完成
2	ADD.D	F4,F0,F2	4	10		13	等待L.D的结果
2	S.D	F4,0（R1）	5	8	14		等待ADD.D的结果
2	DADDIU	R1,R1,#-8	5	9		10	等待ALU
2	BNE	R1,R2,Loop	6	11			等待DADDIU的结果
3	L.D	F0,0（R1）	7	12	13	14	等待BNE完成
3	ADD.D	F4,F0,F2	7	15		18	等待L.D的结果
3	S.D	F4,0（R1）	8	13	19		等待ADD.D的结果
3	DADDIU	R1,R1,#-8	8	14		15	等待ALU
3	BNE	R1,R2,Loop	9	16			等待DADDIU的结果

时钟周期	整型ALU	浮点ALU	数据Cache	CDB
2	1/L.D			
3	1/S.D		1/L.D	
4	1/DADDIU			1/L.D
5		1/ADD.D		1/DADDIU
6				
7	2/L.D			
8	2/S.D		2/L.D	1/ADD.D
9	2/DADDIU		1/S.D	2/L.D
10		2/ADD.D		2/DADDIU
11				
12	3/L.D			

时钟周期	整型ALU	浮点ALU	数据Cache	CDB
13	3/S.D		3/L.D	2/ADD.D
14	3/DADDIU		2/S.D	3/L.D
15		3/ADD.D		3/DADDIU
16				
17				
18				3/ADD.D
19			3/S.D	
20				

扩展：多指令流出-基于动态调度

可以看出：

- 每3个时钟周期就执行一个新循环，每个循环5条指令。

  IPC＝5/3＝1.67条/拍

- 虽然指令的流出率比较高，但是执行效率并不是很高。
  - 16拍共执行15条指令，
  - 平均指令执行速度为15/16＝0.94条/拍。
- 原因是浮点运算少，ALU部件成了瓶颈。

  解决方法：增加一个加法器，把ALU功能和地址运算功能分开。

增加多个整数部件供有效地址计算和ALU操作使用

循环体号	指令	指令发射周期	指令执行周期	访问主存周期	写CDB周期	注释
1	L.D F0, 0(R1)	1	2	3	**4**	第一个发射
1	ADD.D F4, F0, F2	1	5		**8**	等待L.D
1	S.D F4, 0(R1)	2	3	9		等待ADD.D
1	DADDIU R1, R1, #-8	2	**3**		**4**	较早执行
1	BNE R1, R2, Loop	3	5			等待DADDIU
2	L.D F0, 0(R1)	4	6	7	**8**	等待BNE完成
2	ADD.D F4, F0, F2	4	9		12	等待L.D
2	S.D F4, 0(R1)	5	7	13		等待ADD.D
2	DADDIU R1, R1, #-8	5	**6**		7	较早执行
2	BNE R1, R2, Loop	6	8			等待DADDIU
3	L.D F0, 0(R1)	7	9	10	11	等待BNE完成
3	ADD.D F4, F0, F2	7	12		15	等待L.D
3	S.D F4, 0(R1)	8	10	16		等待ADD.D
3	DADDIU R1, R1, #-8	8	**9**		10	较早执行
3	BNE R1, R2, Loop	9	11			等待DADDIU

时钟	整数ALU	地址加法器	浮点ALU	数据缓存	CDB#1	CDB#2
2		1/L.D				
3	1/DADDIU	1/S.D		1/L.D		
4					1/L.D	1/DADDIU
5			1/ADD.D			
6	2/DADDIU	2/L.D				
7		2/S.D		2/L.D	2/DADDIU	
8					1/ADD.D	2/L.D
9	3/DADDIU	3/L.D	2/ADD.D	1/S.D		
10		3/S.D		3/L.D	3/DADDIU	
11					3/L.D	
12			3/ADD.D		2/ADD.D	
13				2/S.D		
14						
15						3/ADD.D
16				3/S.D		

扩展：多指令流出-基于动态调度

- 上述双流出动态调度流水线的性能受限于以下3个因素：
  - 整数部件和浮点部件的工作负载不平衡，没有充分发挥出浮点部件的作用。

    应该设法减少循环中整数型指令的数量。
  - 每个循环叠代中的控制开销太大。
    - 5条指令中有两条指令是辅助指令；
    - 应该设法减少或消除这些指令。
  - 控制相关使得处理机必须等到分支指令的结果出来后才能开始下一条L. D指令的执行。

扩展：多指令流出-基于动态调度

例：考虑如下循环程序在一个双发射处理器上执行过程，第一次不支持前瞻执行，第二次采用了前瞻执行。

```
Loop:  LD      R2, 0(R1)     ；R2为数组元素
       DADDIU  R2, R2, #1    ；R2加1
       SD      R2,  0(R1)    ；保存结果
       DADDIU  R1, R1, #4    ；指针加1
       BNE     R2, R3, Loop
```

- 假设有单独整数功能部件用来进行有效地址计算和分支条件计算
- 假设任意两条指令能在一个周期内提交

不使用前瞻执行

循环体号	指令			发射时刻	执行时刻	内存存取时刻	写CDB时刻	注释
1	LD	R2,	0(R1)	1	2	3	4	第一个发射
1	DADDIU	R2,	R2, #1	1	5		6	等待LD
1	SD	R2,	0(R1)	2	3	7		等待DADDIU
1	DADDIU	R1,	R1, #4	2	3		4	直接执行
1	BNE	R2,	R3, LOOP	3	7			等待DADDIU
2	LD	R2,	0(R1)	4	8	9	10	等待BNE
2	DADDIU	R2,	R2, #1	4	11		12	等待LD
2	SD	R2,	0(R1)	5	9	13		等待DADDIU
2	DADDIU	R1,	R1, #4	5	8		9	等待BNE
2	BNE	R2,	R3, LOOP	6	13			等待DADDIU
3	LD	R2,	0(R1)	7	14	15	16	等待 BNE
3	DADDIU	R2,	R2, #1	7	17		18	等待LD
3	SD	R2,	0(R1)	8	15	19		等待DADDIU
3	DADDIU	R1,	R1, #4	8	14		15	等待BNE
3	BNE	R2,	R3, LOOP	9	19			等待DADDIU

循环体号	指令			发射时刻	执行时刻	内存存取时刻	写CDB时刻	确认时刻	注释
1	LD	R2,	0(R1)	1	2	3	4	5	第一次发射
1	DADDIU	R2,	R2, #1	1	5		6	7	等待LD
1	SD	R2,	0(R1)	2	3			7	等待DADDIU
1	DADDIU	R1,	R1, #4	2	3		4	8	顺序提交
1	BNE	R2,	R3, LOOP	3	7			8	等待DADDIU
2	LD	R2,	0(R1)	4	5	6	7	9	无延迟执行
2	DADDIU	R2,	R2, #1	4	8		9	10	等待LD
2	SD	R2,	0(R1)	5	6			10	等待DADDIU
2	DADDIU	R1,	R1, #4	5	6		7	11	顺序提交
2	BNE	R2,	R3, LOOP	6	10			11	等待DADDIU
3	LD	R2,	0(R1)	7	8	9	10	12	尽可能提前
3	DADDIU	R2,	R2, #1	7	11		12	13	等待LD
3	SD								DIU
3	DAD								前
3	BNE	R2,	R3, LOOP	9	13			14	等待DADDIU

- 分支预测要准确，否则性能将会非常糟糕

  ARM Cortex-A8 错误预测的代价为13个周期

扩展：多指令流出-基于静态调度

- 在典型的超标量处理器中，每个时钟周期可流出1到8条指令。
- 指令按序流出，在流出时进行冲突检测。

  由硬件检测当前流出的指令之间是否存在冲突以及当前流出的指令与正在执行的指令是否有冲突。

举例：一个4-流出的静态调度超标量处理机

- 在取指令阶段，流水线将从取指令部件收到1～4条指令（称为流出包）。
  - 在一个时钟周期内，这些指令有可能是全部都能流出，也可能是只有一部分能流出。
  - 流出部件要负责检查结构冲突和数据冲突，这些流出检查相当复杂，因此发射逻辑将决定最小时钟周期的长度

扩展：多指令流出-基于静态调度

- 流出部件检测结构冲突或者数据冲突。

  在许多静态调度和所有动态调度的超标量处理器中一般分两阶段实现：

  - 第一段：进行流出包内的冲突检测，选出初步判定可以流出的指令；
  - 第二段：检测所选出的指令与正在执行的指令是否有冲突。

扩展：多指令流出-基于静态调度

MIPS处理机是怎样实现超标量的呢？

假设：每个时钟周期流出两条指令：

1条整数型指令＋1条浮点操作指令

- 其中：把load指令、store指令、分支指令归类为整数型指令。

- 这一假设与**HP7100**处理器的组织结构非常相近

扩展：多指令流出-基于静态调度

- 要求：

  同时取两条指令（64位），译码两条指令（64位）。

- 对指令的处理包括以下步骤：
  - 从Cache中取两条指令；
  - 确定哪几条指令可以流出（0～2条指令）；
  - 把它们发送到相应的功能部件。
- 双流出超标量流水线中指令执行的时空图
  - 假设：所有的浮点指令都是加法指令，其执行时间为3个时钟周期。
  - 为简单起见，图中总是把整数指令放在浮点指令的前面。

扩展：多指令流出-基于静态调度

指令类型	流水线工作情况							
整数指令	IF	ID	EX	MEM	WB			
浮点指令	IF	ID	EX	EX	EX	WB		
整数指令		IF	ID	EX	MEM	WB		
浮点指令		IF	ID	EX	EX	EX	WB	
整数指令			IF	ID	EX	MEM	WB	
浮点指令			IF	ID	EX	EX	EX	WB
整数指令				IF	ID	EX	MEM	WB
浮点指令				IF	ID	EX	EX	EX

扩展：多指令流出-基于静态调度

- 采用“1条整数型指令＋1条浮点指令”并行流出的方式，需要增加的硬件很少。
- 浮点load或浮点store指令将使用整数部件，会增加对浮点寄存器的访问冲突。

  增设一个浮点寄存器的读/写端口。

扩展：多指令流出-基于静态调度

- 限制超标量流水线的性能发挥的障碍

  在双发射流水线中要达到吞吐量峰值比在单发射流水线中要困难得多。

  - load指令
    - load后续3条指令都不能使用其结果，否则就会引起停顿。
  - 分支延迟
    - 如果分支指令是流出包中的第一条指令，则其导致3条指令的延迟；
    - 否则就是流出包中的第二条指令，其导致两条指令的延迟。

扩展：多指令流出-基于静态调度

- 为了更有效地提高超标量处理器的并行性，就需要
  - 更有效的编译器
  - 或硬件调度技术

否则超标量处理器收效甚微。

3、静态调度 - 指令调度

指令调度的基本方法

- 指令调度：找出不相关的指令序列，让它们在流水线上重叠并行执行。
  - 为了避免流水线的暂停，要事先找出指令代码中相关的指令并加以分离，使其相隔的时钟周期正好等于原来指令在流水线中的延迟时间。
- 制约编译器指令调度的因素
  - 程序固有的指令级并行性
  - 流水线功能部件的延迟

3、静态调度 - 指令调度

本节使用的浮点流水线的延迟

产生结果的指令	使用结果的指令	延迟(cycles)
浮点计算	另一个浮点计算	3
浮点计算	浮点store(S.D)	2
浮点Load(L.D)	浮点计算	1
浮点Load(L.D)	浮点store(S.D)	0

3、静态调度 - 指令调度

例 对于下面的源代码，转换成MIPS汇编语言，在不进行指令调度和进行指令调度两种情况下，分析其代码一次循环所需的执行时间。

```
for (i=1000; i>0; i--)
    x[i] = x[i] + s;
```

解：

先把该程序翻译成MIPS汇编语言代码

```
Loop:   L.D     F0, 0(R1)
        ADD.D   F4, F0, F2
        S.D     F4, 0(R1)
        DADDIU  R1, R1, #-8
        BNE     R1, R2, Loop
```

3、静态调度 - 指令调度

- 在不进行指令调度的情况下，根据表中给出的浮点流水线中指令执行的延迟，程序的实际执行情况如下：

			指令流出时钟
Loop:	L.D	F0, 0(R1)	1
	(暂停)		2
	ADD.D	F4, F0, F2	3
	(暂停)		4
	(暂停)		5
	S.D	F4, 0(R1)	6
	DADDIU	R1, R1, #-8	7
	(暂停)		8
	BNE	R1, R2, Loop	9
	(暂停)		10

3、静态调度 - 指令调度

- 在用编译器对上述程序进行指令调度以后，程序的执行情况如下：

			指令流出时钟
Loop:	L.D	F0, 0(R1)	1
	DADDIU	R1, R1, #-8	2
	ADD.D	F4, F0, F2	3
	(暂停)		4
	BNE	R1, R2, Loop	5
	S.D	F4, 8(R1)	6

3、静态调度 - 指令调度

进一步分析：

- 怎样提高整个执行过程中有效操作的比率？
  - 每6个时钟周期完成一个循环体，而其中针对数组元素的实际操作（load、ADD和store）只用了3个时钟周期，另外3个时钟周期是循环开销——DADDUI和BNE及一次暂停
  - 要想减少这3个时钟周期的影响，在循环体中需要有更多循环开销指令之外的其他有效操作

3、静态调度 - 循环展开

- 循环展开
  - 把循环体的代码复制多次并按顺序排放， 然后相应调整循环的结束条件。
  - 开发循环级并行的有效方法
    - **消去了分支操作，从而使不同循环体的指令能被调度到一起**
    - **相对于分支转移和循环开销指令之外，就有更多的有效指令在一个循环体中执行**

3、静态调度 - 循环展开

例 将前例中的循环展开3次得到4个循环体，然后对展开后的指令序列在不调度和调度两种情况下，分析代码的性能。假定R1的初值为32的倍数，即循环次数为4的倍数。消除冗余的指令，并且不要重复使用寄存器。

3、静态调度 - 循环展开

① 循环展开和指令调度是如何影响数据相关的

- 去掉了多余的分支指令

```
Loop:  L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8      ; 去掉 BNE
       L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8      ; 去掉 BNE
       L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8      ; 去掉 NBE
       L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8
       BNE      R1,R2,Loop
```

3、静态调度 - 循环展开

① 循环展开和指令调度是如何影响数据相关的

- 去掉了多余的分支指令

可由编译器解决如下：把对各个R1的计算折合到指令L.D和S.D的位移量中，并在最后用指令DADDUI减去分量32，这使得编译器能够除去另外3条多余的DADDUI指令。

```
Loop:  L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8      ; 去掉 BNE
       L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8      ; 去掉 BNE
       L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8      ; 去掉 NBE
       L.D      F0,0(R1)
       ADD.D    F4,F0,F2
       S.D      F4,0(R1)
       DADDUI   R1,R1,#-8
       BNE      R1,R2,Loop
```

3、静态调度 - 循环展开

② 除去多余循环开销

```
Loop:   L.D      F0,0(R1)
        ADD.D    F4,F0,F2
        S.D      F4,0(R1)      ; 去掉 DADDUI 和 BNE
        L.D      F0,-8(R1)
        ADD.D    F4,F0,F2
        S.D      F4,-8(R1)     ; 去掉 DADDUI 和 BNE
        L.D      F0,-16(R1)
        ADD.D    F4,F0,F2
        S.D      F4,-16(R1)    ; 去掉 DADDUI 和 BNE
        L.D      F0,-24(R1)
        ADD.D    F4,F0,F2
        S.D      F4,-24(R1)
        DADDUI   R1,R1,#-32
        BNE      R1,R2,Loop
```

3、静态调度 - 循环展开

③ 存在的数据相关

- 名相关
- 数据相关

```
Loop:   L.D      F0,0(R1)
        ADD.D    F4,F0,F2
        S.D      F4,0(R1)       ; 去掉 DADDUI 和 BNE
        L.D      F0,-8(R1)
        ADD.D    F4,F0,F2
        S.D      F4,-8(R1)      ; 去掉 DADDUI 和 BNE
        L.D      F0,-16(R1)
        ADD.D    F4,F0,F2
        S.D      F4,-16(R1)     ; 去掉 DADDUI 和 BNE
        L.D      F0,-24(R1)
        ADD.D    F4,F0,F2
        S.D      F4,-24(R1)
        DADDUI   R1,R1,#-32
        BNE      R1,R2,Loop
```

3、静态调度 - 循环展开

```
Loop:   L.D      F0,0(R1)
        ADD.D    F4,F0,F2
        S.D      F4,0(R1)
        L.D      F6,-8(R1)
        ADD.D    F8,F6,F2
        S.D      F8,-8(R1)
        L.D      F10,-16(R1)
        ADD.D    F12,F10,F2
        S.D      F12,-16(R1)
        L.D      F14,-24(R1)
        ADD.D    F16,F14,F2
        S.D      F16,-24(R1)
        DADDUI   R1,R1,#-32
        BNE      R1,R2,Loop
```

④ 去掉了名相关

3、静态调度 - 循环展开

- 展开后没有调度的代码如下(需要分配寄存器)

			指令流出时钟
Loop:	L.D	F0, 0(R1)	1
	（暂停）		2
	ADD.D	F4, F0, F2	3
	（暂停）		4
	（暂停）		5
	S.D	F4, 0(R1)	6
	L.D	F6, -8(R1)	7
	（暂停）		8
	ADD.D	F8, F6, F2	9
	（暂停）		10
	（暂停）		11
	S.D	F8, -8(R1)	12
	L.D	F10, -16(R1)	13
	（暂停）		14

			指令流出时钟
	ADD.D	F12, F10, F2	15
	（暂停）		16
	（暂停）		17
	S.D	F12, -16（R1）	18
	L.D	F14, -24（R1）	19
	（暂停）		20
	ADD.D	F16, F14, F2	21
	（暂停）		22
	（暂停）		23
	S.D	F16, -24（R1）	24
	DADDIU	R1, R1, # -32	25
	（暂停）		26
	BNE	R1, R2, Loop	27
	（暂停）		28

50%是暂停周期！

3、静态调度 - 循环展开

			指令流出时钟
Loop:	L.D	F0, 0（R1）	1
	L.D	F6, -8（R1）	2
	L.D	F10, -16（R1）	3
	L.D	F14, -24（R1）	4
	ADD.D	F4, F0, F2	5
	ADD.D	F8, F6, F2	6
	ADD.D	F12, F10, F2	7
	ADD.D	F16, F14, F2	8
	S.D	F4, 0（R1）	9
	S.D	F8, -8（R1）	10
	DADDIU	R1, R1, # -32	12
	S.D	F12, 16（R1）	11
	BNE	R1, R2, Loop	13
	S.D	F16, 8（R1）	14

⑤ 去除数据相关

3、静态调度 - 循环展开

- 调度后的代码如下：

			指令流出时钟
Loop:	L.D	F0, 0（R1）	1
	L.D	F6, -8（R1）	2
	L.D	F10, -16（R1）	3
	L.D	F14, -24（R1）	4
	ADD.D	F4, F0, F2	5
	ADD.D	F8, F6, F2	6
	ADD.D	F12, F10, F2	7
	ADD.D	F16, F14, F2	8
	S.D	F4, 0（R1）	9
	S.D	F8, -8（R1）	10
	DADDIU	R1, R1, # -32	12
	S.D	F12, 16（R1）	11
	BNE	R1, R2, Loop	13
	S.D	F16, 8（R1）	14

没有暂停周期！

结论：通过循环展开、寄存器重命名和指令调度，可以有效开发出指令级并行。

3、静态调度 - 循环展开和指令调度

- 循环展开和指令调度的注意事项 （必不可少的决策及变换）
  - ① 判断出展开独立不相关（除了维持循环结构的代码）的循环体，将会对程序运行性能提高有很大的帮助。
  - ② 为了避免不同的计算用到相同寄存器所发生的不必要限制，需要使用不同的寄存器。
  - ③ 消去额外的测试指令和分支指令，调整维持循环体运行的代码。

3、静态调度 - 循环展开和指令调度

2. 循环展开和指令调度的注意事项 （必不可少的决策及变换）

④ 找出不同循环体之间独立不相关的load指令和store指令，从而可以对展开之后的循环程序中的load指令和store指令做交换操作。要做到这些，需要对引用的内存地址进行分析并发现上述指令指向了不同的内存地址。

⑤ 调度优化代码，保留那些为得到正确结果而必须保持的相关关系。

⑥ 知道把S.D指令移到DADDUI和BNE指令之后是合法有效的，且必须正确调整S.D指令中的位移量

3、静态调度 - 循环展开和指令调度

例：在静态多发射超标量处理器中

使用循环展开和流水线调度

考察一个双发射、静态调度的超标量MIPS流水线

- 流水线延迟的假设如前
- 一个周期可以发射两条指令
  - **一条整型（包括load、store、分支）**
  - **一条浮点**

展开并调度上一节的循环程序

- 为了无延迟地调度该循环程序，必须对该程序展开5个循环体

	定点指令		浮点指令	时钟周期
Loop:	L.D	F0,0(R1)		1
	L.D	F6,-8(R1)		2
	L.D	F10,-16(R1)	ADD.D F4,F0,F2	3
	L.D	F14,-24(R1)	ADD.D F8,F6,F2	4
	L.D	F18,-32(R1)	ADD.D F12,F10,F2	5
	S.D	F4,0(R1)	ADD.D F16,F14,F2	6
	S.D	F8,-8(R1)	ADD.D F20,F18,F2	7
	S.D	F12,-16(R1)		8
	DADDUI	R1,R1,#-40		9
	S.D	F16,16(R1)		10
	BNE	R1,R2,Loop		11
	S.D	F20,8(R1)		12

2.4个周期执行一次单位循环

3、静态调度 - 循环展开和指令调度

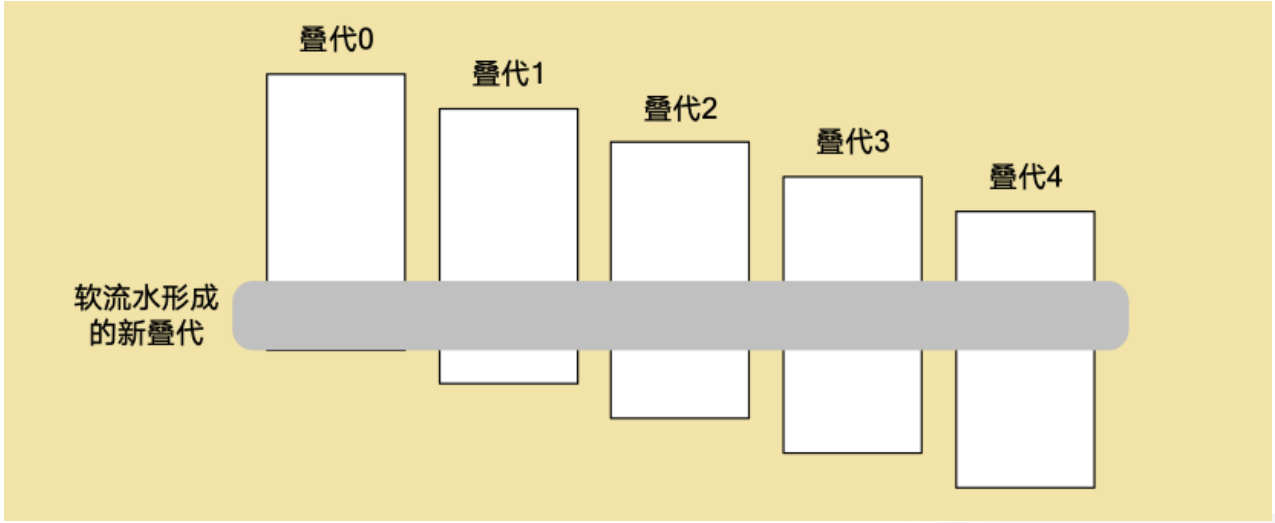
3. 循环展开所受到的制约因素：

- 循环开销的减少程度
  - 当循环被展开4次以后，在14个时钟周期里，循环的开销只占用了2个周期，如果循环展开8次...
- 代码量的大小
  - 代码量的增长可能会受到存储空间的限制，或者增加指令cache的缺失率
- 编译器的限制
  - 过分地展开和调度循环可能会引起寄存器不够用，这被称为寄存器压力
  - 复杂的高层变换使得编译器越来越复杂了，而实际效果是很难衡量的

3、静态调度 – 软流水

简介

- 软流水技术的核心思想是从循环不同的叠代中抽取一部分指令（循环控制指令除外）拼成一个新的循环叠代。
- 目的
  - 将同一叠代中的相关指令分布到不同的叠代中
  - 将不同叠代中的相关指令封装到同一叠代中



3、静态调度 – 软流水

➢ 实例分析

例 试用软流水技术处理前例中的循环，假设数组x有n个元素。

解 软流水需要从原循环的多个叠代中选择指令拼成新的循环，因此我们首先将原循环展开。

```
叠代i:      ; 修改x[i]并保存
            L.D     F0, 0(R1)
            ADD.D   F4, F0, F2
            S.D     F4, 0(R1)
叠代i+1:    ; 修改x[i-1]并保存
            L.D     F0, 0(R1)
            ADD.D   F4, F0, F2
            S.D     F4, 0(R1)
叠代i+2:    ; 修改x[i-2]并保存
            L.D     F0, 0(R1)
            ADD.D   F4, F0, F2
            S.D     F4, 0(R1)
```

3、静态调度 – 软流水

➢ 实例分析

例 试用软流水技术处理前例中的循环，假设数组x有n个元素。

解 软流水需要从原循环的多个叠代中选择指令拼成新的循环，因此我们首先将原循环展开。

```
叠代i:      ; 修改x[i]并保存
            L.D     F0, 0(R1)
            ADD.D   F4, F0, F2
            S.D     F4, 0(R1)
叠代i+1:    ; 修改x[i-1]并保存
            L.D     F0, 0(R1)
            ADD.D   F4, F0, F2
            S.D     F4, 0(R1)
叠代i+2:    ; 修改x[i-2]并保存
            L.D     F0, 0(R1)
            ADD.D   F4, F0, F2
            S.D     F4, 0(R1)
```

3、静态调度 – 软流水

从这3个叠代中分别选出一条指令，与原有的循环控制指令拼在一起得到一个新的叠代。

```
        DADDUI R1, R1, #-16          // I1：R1保存x[n-2]的地址
        L.D    F0, 16（R1）          // I2：取x[n]
        ADD.D F4, F0, F2             // I3：x[n] = x[n] + F2
        L.D    F0, 8（R1）           // I4：取x[n-1]
Loop:   S.D    F4, 16（R1）          // I5：存x[i+2]
        ADD.D F4, F0, F2             // I6：x[i+1] = x[i+1] + F2
        L.D    F0, 0（R1）           // I7：取x[i]
        BNE    R1, R2, Loop          // I8
        DADDUI  R1, R1, #-8          // I9：填充分支延迟槽
        S.D    F4, 8（R1）           // I10：存x[2]
        ADD.D F4, F0, F2             // I11：x[1] = x[1] + F2
        S.D    F4, 0（R1）           // I12：存x[1]
```

3、静态调度 – 软流水

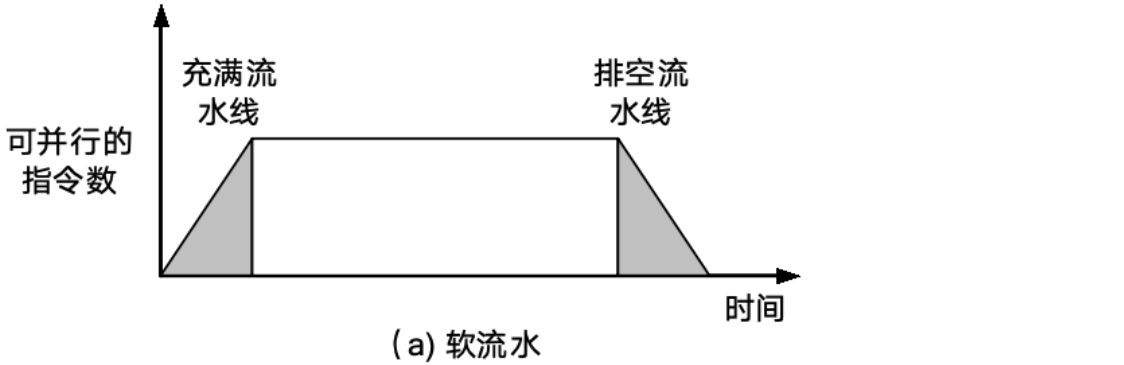
分析：

- 新循环的结束条件仍为：R1=R2
- 新循环从元素x[n-2]开始从头处理的，元素x[n]与x[n-1]只是从半途开始处理。
- 新循环的最后一次叠代只处理完元素x[3]，x[2]刚被修改完结果尚未写回，x[1] 刚被取出，需要被修改并写回。
- 新循环的每个叠代相当于流水执行了原循环的3个叠代。元素x[n/n-1/2/1]的处理相当于充满和排空这条“流水线”。

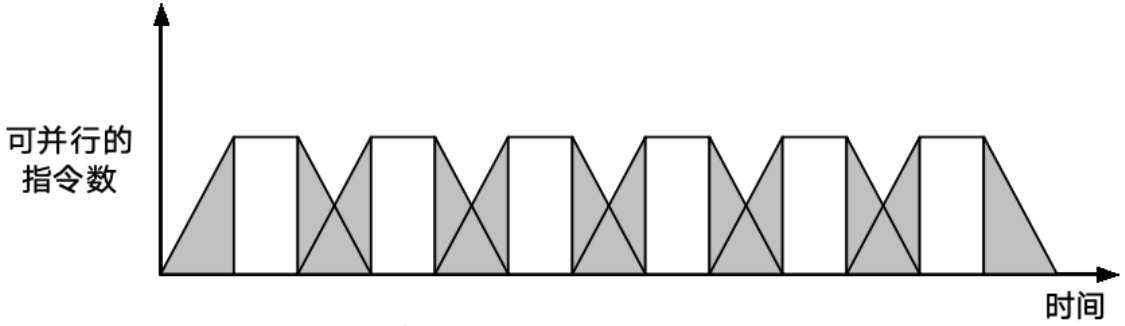
3、静态调度 – 软流水

性能比较：软流水 vs. 循环展开

- 循环展开主要减少由分支指令和修改循环索引变量的指令所引起的循环控制开销。
- 软流水使叠代内的指令级并行达到最大。



(a) 软流水



(b) 循环展开

总结一下

指令多流出处理器受哪些因素的限制呢？

主要受以下3个方面的影响：

- 程序所固有的指令级并行性；
- 硬件实现上的困难；
- 超标量和超长指令字处理器固有的技术限制。

➢ 设计多流出处理器的难点：

- 访存开销
- 硬件复杂性
- 编译器技术



授课教师：王强
邮箱：qiang.wang@hit.edu.cn

谢谢

Thank You